

NFL Draft prediction manual Bayesian

DS 4420 Andrew Thomas Lotocki and Alexander Rees

1. Shared helpers

```
library(caret)
data_dir <- "data"

prep_train_test <- function(csv_name, predictor_cols, split_seed = 42) {
  csv_data <- read.csv(file.path(data_dir, csv_name), stringsAsFactors = FALSE, check.names = FALSE)
  player_labels <- if ("Player" %in% names(csv_data)) {
    as.character(csv_data$Player)
  } else {
    paste0("Obs_", seq_len(nrow(csv_data)))
  }
  model_frame <- csv_data[, c("Drafted", predictor_cols), drop = FALSE]
  if ("Height" %in% names(model_frame)) {
    model_frame$Height <- suppressWarnings(as.numeric(as.character(model_frame$Height)))
  }
  row_is_complete <- complete.cases(model_frame)
  model_frame <- model_frame[row_is_complete, , drop = FALSE]
  player_labels <- player_labels[row_is_complete]
  drafted_binary <- as.integer(model_frame$Drafted)
  predictor_matrix <- as.matrix(model_frame[, predictor_cols, drop = FALSE])
  set.seed(split_seed)
  n_players <- nrow(predictor_matrix)
  train_index <- sample.int(n_players, floor(0.8 * n_players))
  test_index <- setdiff(seq_len(n_players), train_index)
  predictors_train_only <- predictor_matrix[train_index, , drop = FALSE]
  train_column_sd <- apply(predictors_train_only, 2, sd)
  train_column_sd[train_column_sd == 0 | is.na(train_column_sd)] <- 1
  design_matrix <- cbind(
    1,
    scale(predictor_matrix, center = colMeans(predictors_train_only), scale = train_column_sd)
  )
  colnames(design_matrix) <- c("bias", predictor_cols)
  list(
    X_train = design_matrix[train_index, , drop = FALSE],
    y_train = drafted_binary[train_index],
    X_test = design_matrix[test_index, , drop = FALSE],
    y_test = drafted_binary[test_index],
    player_test = player_labels[test_index],
    pred_cols = predictor_cols,
    n = n_players
  )
}
```

```

sigmoid <- function(linear_predictor) {
  ifelse(
    linear_predictor >= 0,
    1 / (1 + exp(-linear_predictor)),
    exp(linear_predictor) / (1 + exp(linear_predictor))
  )
}

log_likelihood_logit <- function(coefficients, design_matrix, drafted_binary) {
  linear_predictor <- as.vector(design_matrix %*% coefficients)
  sum(drafted_binary * linear_predictor - log1p(exp(linear_predictor)))
}

log_prior <- function(coefficients, prior_variance_scale) {
  -0.5 * sum(coefficients^2) / prior_variance_scale
}

log_posterior <- function(coefficients, design_matrix, drafted_binary, prior_variance_scale) {
  log_likelihood_logit(coefficients, design_matrix, drafted_binary) +
  log_prior(coefficients, prior_variance_scale)
}

run_mh <- function(design_matrix, drafted_binary, n_iter, proposal_sd, prior_variance_scale, seed = 42) {
  set.seed(seed)
  n_coefficients <- ncol(design_matrix)
  current_coefficients <- rep(0, n_coefficients)
  mcmc_sample_matrix <- matrix(NA_real_, nrow = n_iter, ncol = n_coefficients)
  colnames(mcmc_sample_matrix) <- colnames(design_matrix)
  log_posterior_current <- log_posterior(
    current_coefficients, design_matrix, drafted_binary, prior_variance_scale
  )
  n_accepted <- 0L

  for (iteration in seq_len(n_iter)) {
    proposed_coefficients <- current_coefficients + rnorm(n_coefficients, mean = 0, sd = proposal_sd)
    log_posterior_proposed <- log_posterior(
      proposed_coefficients, design_matrix, drafted_binary, prior_variance_scale
    )
    if (log(stats::runif(1)) < log_posterior_proposed - log_posterior_current) {
      current_coefficients <- proposed_coefficients
      log_posterior_current <- log_posterior_proposed
      n_accepted <- n_accepted + 1L
    }
    mcmc_sample_matrix[iteration, ] <- current_coefficients
  }

  acceptance_rate <- as.numeric(n_accepted) / n_iter
  cat(sprintf("Acceptance rate: %.3f\n", acceptance_rate))
  list(chain = mcmc_sample_matrix, acceptance_rate = acceptance_rate)
}

thin_chain <- function(full_chain, burn_in, thin) {
  full_chain[seq(burn_in + 1, nrow(full_chain), by = thin), , drop = FALSE]
}

```

```

}

posterior_quantile_table <- function(posterior_draws, probs = c(0.25, 0.5, 0.75)) {
  coef_quantiles <- apply(posterior_draws, 2, stats::quantile, probs = probs)
  data.frame(
    coefficient = colnames(posterior_draws),
    posterior_mean = colMeans(posterior_draws),
    q1 = as.numeric(coef_quantiles[1, ]),
    q2 = as.numeric(coef_quantiles[2, ]),
    q3 = as.numeric(coef_quantiles[3, ])
  )
}

plot_chain_acf <- function(parameter_trace, parameter_name = "parameter") {
  par(mfrow = c(1, 2), mar = c(4, 4, 2, 1))
  plot(parameter_trace, type = "l", main = sprintf("Convergence of %s", parameter_name),
    xlab = "Iteration", ylab = parameter_name)
  abline(h = mean(parameter_trace), col = "red", lty = 2)
  acf(parameter_trace, main = sprintf("ACF: %s", parameter_name))
  par(mfrow = c(1, 1))
}

posterior_mean_prob_drafted <- function(design_matrix, posterior_draws) {
  linear_predictors <- design_matrix %*% t(posterior_draws)
  rowMeans(sigmoid(linear_predictors))
}

# Metrics
classification_metrics <- function(y_true, y_hat) {
  reference_factor <- factor(as.integer(y_true), levels = c(0L, 1L))
  predicted_factor <- factor(as.integer(y_hat), levels = c(0L, 1L))
  confusion_mat <- confusionMatrix(predicted_factor, reference_factor, positive = "1", mode = "everything")
  metrics_by_class <- confusion_mat$byClass
  list(
    accuracy = unname(confusion_mat$overall["Accuracy"]),
    precision = unname(metrics_by_class["Precision"]),
    recall = unname(metrics_by_class["Recall"]),
    f1 = unname(metrics_by_class["F1"])
  )
}

plot_test_dashboard <- function(y_test, probs, y_hat, player_names, threshold, main_title) {
  actual_drafted <- as.integer(y_test)
  predicted_drafted <- as.integer(y_hat)
  count_true_negative <- sum(actual_drafted == 0 & predicted_drafted == 0)
  count_false_negative <- sum(actual_drafted == 1 & predicted_drafted == 0)
  count_false_positive <- sum(actual_drafted == 0 & predicted_drafted == 1)
  count_true_positive <- sum(actual_drafted == 1 & predicted_drafted == 1)
  par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
  plot(0, 0, type = "n", xlim = c(0, 2), ylim = c(0, 2), asp = 1,
    xlab = "Actual", ylab = "Predicted", main = "Confusion",
    xaxt = "n", yaxt = "n")
  rect(0, 1, 1, 2, col = "wheat"); text(0.5, 1.5, count_true_negative, cex = 2)

```

```

rect(1, 1, 2, 2, col = "wheat"); text(1.5, 1.5, count_false_negative, cex = 2)
rect(0, 0, 1, 1, col = "wheat"); text(0.5, 0.5, count_false_positive, cex = 2)
rect(1, 0, 2, 1, col = "wheat"); text(1.5, 0.5, count_true_positive, cex = 2)
axis(1, at = c(0.5, 1.5), labels = c("Not drafted", "Drafted"))
axis(2, at = c(0.5, 1.5), labels = c("Pred: drafted", "Pred: not"), las = 1)
test_row_index <- seq_along(probs)
plot(test_row_index, probs, pch = 19, col = ifelse(actual_drafted == 1, "darkgreen", "red"),
      xlab = "Test player (row)", ylab = "P(drafted)", main = main_title)
abline(h = threshold, lty = 2)
legend("topleft", legend = c("Was drafted", "Not drafted"), col = c("darkgreen", "red"), pch = 19, bty = "n")
for (j in which(actual_drafted == 0 & predicted_drafted == 1)) {
  text(test_row_index[j], probs[j], player_names[j], pos = 3, cex = 0.55)
}
for (j in which(actual_drafted == 1 & predicted_drafted == 0)) {
  text(test_row_index[j], probs[j], player_names[j], pos = 1, cex = 0.55)
}
par(mfrow = c(1, 1))
}

```

2. Fitting all positions

```

default_tau_sq <- 1
default_prob_threshold <- 0.4

n_iter <- 25000
burn_in <- 5000
thin <- 5

position_config <- list(
  WR = list(
    file = "wr_data.csv",
    proposal_sd = 0.08,
    tau_sq = default_tau_sq,
    prob_threshold = default_prob_threshold,
    preds = c(
      "Height", "Weight", "40yd", "Vertical",
      "receiving_YDS", "receiving_TD", "receiving_YPR", "receiving_REC"
    )
  ),
  QB = list(
    file = "qb_data.csv",
    proposal_sd = 0.14,
    tau_sq = 0.5,
    prob_threshold = 0.45,
    preds = c(
      "Height", "Weight", "40yd", "Vertical",
      "passing_ATT", "passing_TD", "passing_YDS", "passing_YPA"
    )
  ),
  RB = list(
    file = "rb_data.csv",
    proposal_sd = 0.10,

```

```

    tau_sq = default_tau_sq,
    prob_threshold = 0.5,
    preds = c(
      "Height", "Weight", "40yd", "Vertical",
      "rushing_CAR", "rushing_YDS", "rushing_TD", "rushing_YPC"
    )
  ),
  DE = list(
    file = "de_data.csv",
    proposal_sd = 0.21,
    tau_sq = default_tau_sq,
    prob_threshold = 0.55,
    preds = c(
      "Height", "Weight", "40yd", "Vertical", "Bench",
      "defensive_SACKS", "defensive_TOT", "defensive_TFL"
    )
  ),
  CB = list(
    file = "cb_data.csv",
    proposal_sd = 0.12,
    tau_sq = default_tau_sq,
    prob_threshold = 0.55,
    preds = c(
      "Height", "Weight", "40yd", "Vertical", "Bench",
      "defensive_PD", "defensive_TOT", "interceptions_INT"
    )
  )
)

base_seed <- 42
fits <- list()
compare_rows <- list()

for (i in seq_along(position_config)) {
  pos <- names(position_config)[i]
  cfg <- position_config[[pos]]
  prep <- prep_train_test(cfg$file, cfg$preds, split_seed = 42 + i)
  tau_pos <- cfg$tau_sq
  thr <- cfg$prob_threshold
  mh <- run_mh(
    prep$X_train, prep$y_train,
    n_iter, cfg$proposal_sd, tau_pos,
    seed = base_seed + i
  )
  S <- thin_chain(mh$chain, burn_in = burn_in, thin = thin)

  ptr <- posterior_mean_prob_drafted(prep$X_train, S)
  pte <- posterior_mean_prob_drafted(prep$X_test, S)
  y_hat_train <- as.integer(ptr >= thr)
  y_hat_test <- as.integer(pte >= thr)

  m_tr <- classification_metrics(prep$y_train, y_hat_train)
  m_te <- classification_metrics(prep$y_test, y_hat_test)
}

```

```

fits[[pos]] <- list(
  prep = prep,
  samples = S,
  acceptance_rate = mh$acceptance_rate,
  post_prob_test = pte,
  y_hat_train = y_hat_train,
  y_hat_test = y_hat_test,
  tau_sq = tau_pos,
  prob_threshold = thr,
  metrics_train = m_tr,
  metrics_test = m_te
)

compare_rows[[pos]] <- data.frame(
  position = pos,
  n = prep$n,
  n_train = nrow(prep$X_train),
  n_test = nrow(prep$X_test),
  p = ncol(prep$X_train),
  tau_sq = tau_pos,
  prob_threshold = thr,
  proposal_sd = cfg$proposal_sd,
  acceptance_rate = mh$acceptance_rate,
  acc_train = m_tr$accuracy,
  acc_test = m_te$accuracy,
  recall_train = m_tr$recall,
  recall_test = m_te$recall,
  f1_train = m_tr$f1,
  f1_test = m_te$f1,
  precision_train = m_tr$precision,
  precision_test = m_te$precision
)
}

```

```

## Acceptance rate: 0.317
## Acceptance rate: 0.370
## Acceptance rate: 0.352
## Acceptance rate: 0.382
## Acceptance rate: 0.386

```

```

compare_tbl <- do.call(rbind, compare_rows)
rownames(compare_tbl) <- NULL

```

Comparison across positions

```

tune_tbl <- compare_tbl[, c(
  "position", "n", "n_train", "n_test", "p",
  "tau_sq", "prob_threshold", "proposal_sd", "acceptance_rate"
)]
tune_tbl$tau_sq <- round(tune_tbl$tau_sq, 3)

```

```

tune_tbl$prob_threshold <- round(tune_tbl$prob_threshold, 3)
tune_tbl$proposal_sd <- round(tune_tbl$proposal_sd, 3)
tune_tbl$acceptance_rate <- round(tune_tbl$acceptance_rate, 3)
tune_tbl$acceptance_pct <- paste0(round(100 * compare_tbl$acceptance_rate, 1), "%")

knitr::kable(
  tune_tbl,
  col.names = c(
    "Position", "n", "n_train", "n_test", "p",
    "Prior tau_sq", "Prob. threshold", "Proposal SD", "Accept. rate", "Accept. rate (%)"
  ),
  caption = "Per-position tuning",
  align = "lrrrrrrrrrr"
)

```

Table 1: Per-position tuning

Position	n	n_train	n_test	p	Prior tau_sq	Prob. threshold	Proposal SD	Accept. rate	Accept. rate (%)
WR	558	446	112	9	1.0	0.40	0.08	0.317	31.7%
QB	212	169	43	9	0.5	0.45	0.14	0.370	37%
RB	338	270	68	9	1.0	0.50	0.10	0.352	35.2%
DE	71	56	15	9	1.0	0.55	0.21	0.382	38.2%
CB	211	168	43	9	1.0	0.55	0.12	0.386	38.6%

```

perf_tbl <- data.frame(
  Position = compare_tbl$position,
  train_pct_correct = paste0(round(100 * compare_tbl$acc_train, 1), "%"),
  test_pct_correct = paste0(round(100 * compare_tbl$acc_test, 1), "%"),
  train_recall = round(compare_tbl$recall_train, 3),
  test_recall = round(compare_tbl$recall_test, 3),
  train_f1 = round(compare_tbl$f1_train, 3),
  test_f1 = round(compare_tbl$f1_test, 3),
  train_precision = round(compare_tbl$precision_train, 3),
  test_precision = round(compare_tbl$precision_test, 3)
)

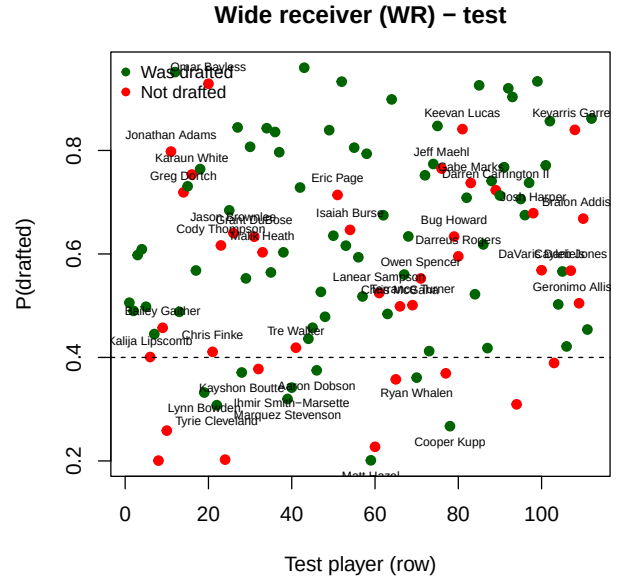
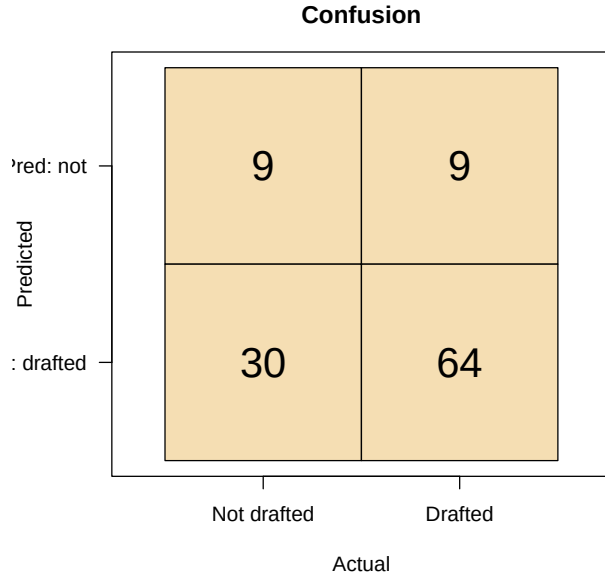
knitr::kable(
  perf_tbl,
  col.names = c(
    "Position",
    "Train % correct", "Test % correct",
    "Train recall", "Test recall",
    "Train F1", "Test F1",
    "Train precision", "Test precision"
  ),
  caption = "Train/test metrics.",
  align = "lrrrrrrrrrr"
)

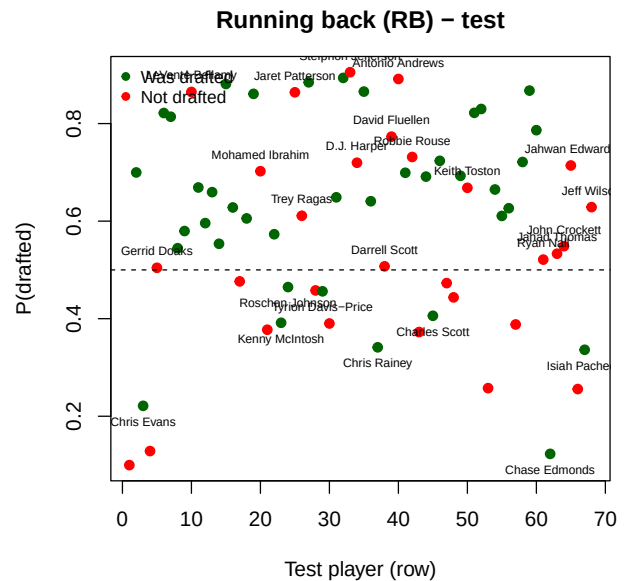
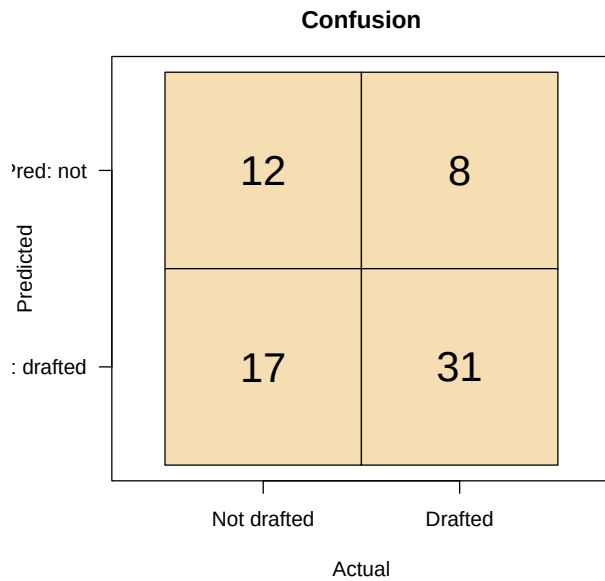
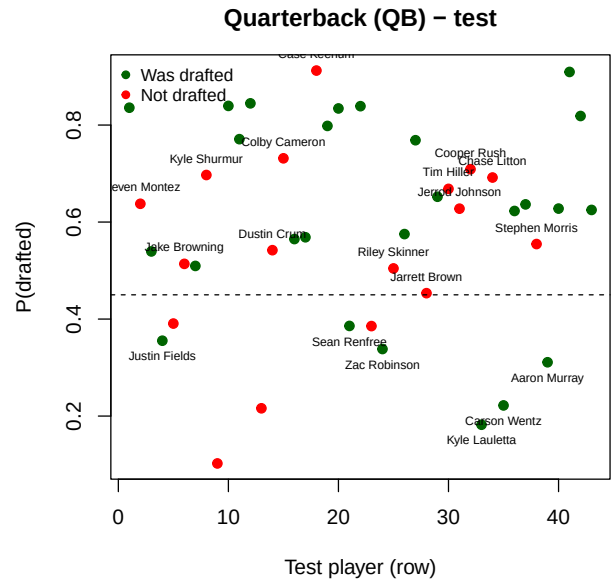
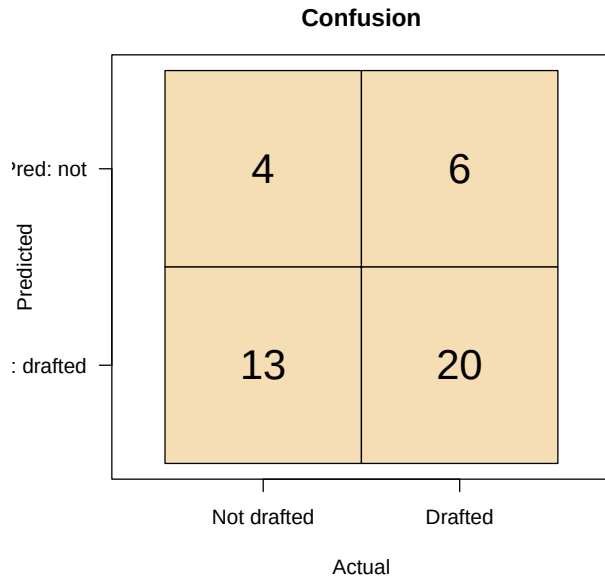
```

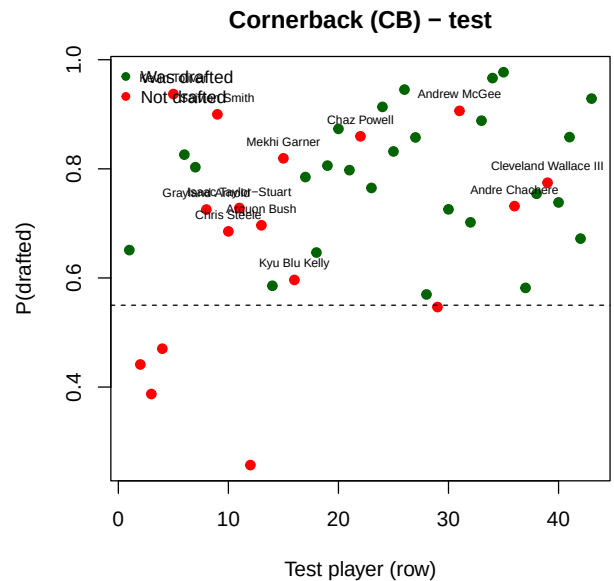
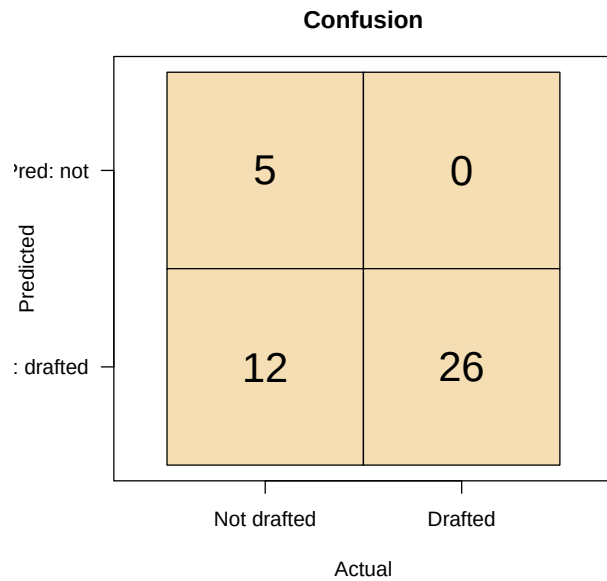
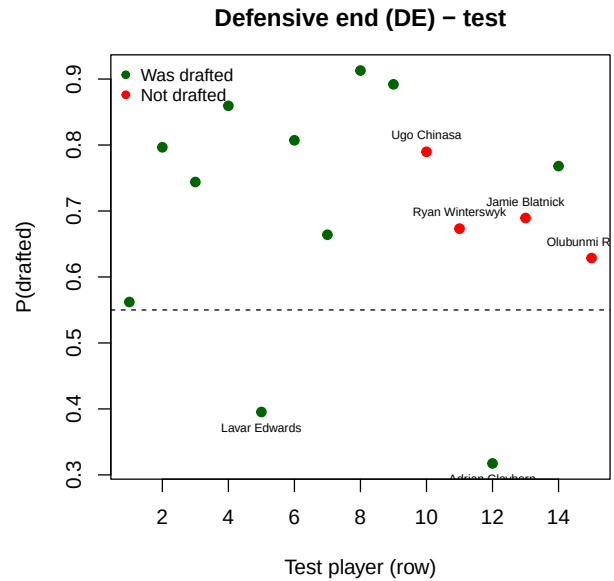
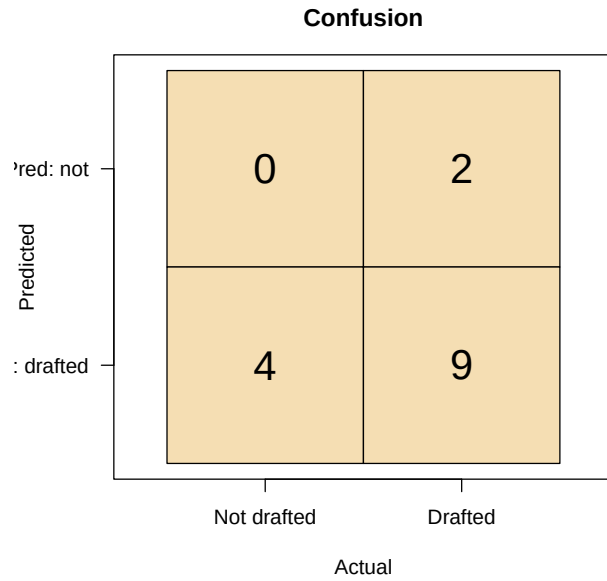
Table 2: Train/test metrics.

Position	Train % correct	Test % correct	Train recall	Test recall	Train F1	Test F1	Train precision	Test precision
WR	66.1%	65.2%	0.879	0.877	0.749	0.766	0.652	0.681
QB	74%	55.8%	0.944	0.769	0.823	0.678	0.729	0.606
RB	71.5%	63.2%	0.805	0.795	0.769	0.713	0.736	0.646
DE	76.8%	60%	0.902	0.818	0.851	0.750	0.804	0.692
CB	79.8%	72.1%	0.976	1.000	0.879	0.813	0.800	0.684

```
pos_labels <- c(
  WR = "Wide receiver", QB = "Quarterback", RB = "Running back",
  DE = "Defensive end", CB = "Cornerback"
)
for (pos in names(position_config)) {
  fit <- fits[[pos]]
  pr <- fit$prep
  plot_test_dashboard(
    pr$y_test, fit$post_prob_test, fit$y_hat_test, pr$player_test,
    fit$prob_threshold, paste0(pos_labels[[pos]], " (", pos, ") - test")
  )
}
```

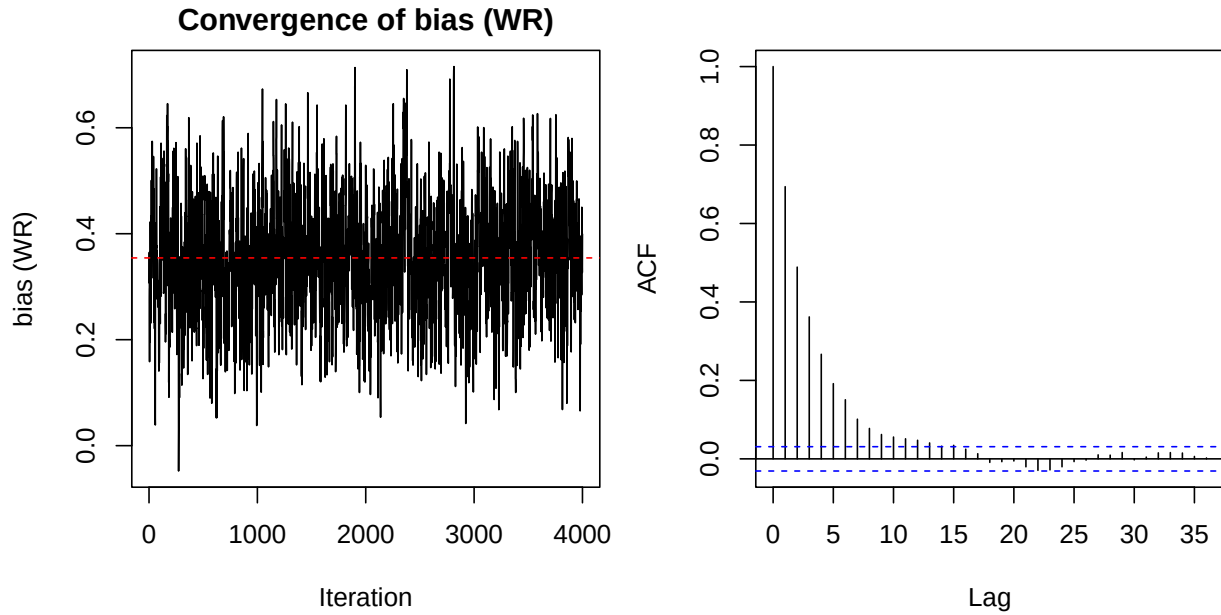






3. WR

```
S <- fits$WR$samples
pr <- fits$WR$prep
plot_chain_acf(S[, "bias"], parameter_name = "bias (WR)")
```



```
knitr::kable(
  posterior_quantile_table(S),
  digits = 3,
  caption = "WR: posterior summaries"
)
```

Table 3: WR: posterior summaries

	coefficient	posterior_mean	q1	q2	q3
bias	bias	0.354	0.282	0.353	0.428
Height	Height	-0.134	-0.291	-0.134	0.028
Weight	Weight	0.128	-0.033	0.129	0.289
40yd	40yd	-0.134	-0.221	-0.131	-0.049
Vertical	Vertical	0.142	0.057	0.144	0.225
receiving_YDS	receiving_YDS	0.565	0.312	0.571	0.820
receiving_TD	receiving_TD	0.393	0.262	0.389	0.523
receiving_YPR	receiving_YPR	-0.044	-0.156	-0.045	0.062
receiving_REC	receiving_REC	0.024	-0.221	0.021	0.261

```
cat("WR test confusion (row = actual, col = predicted):\n")
```

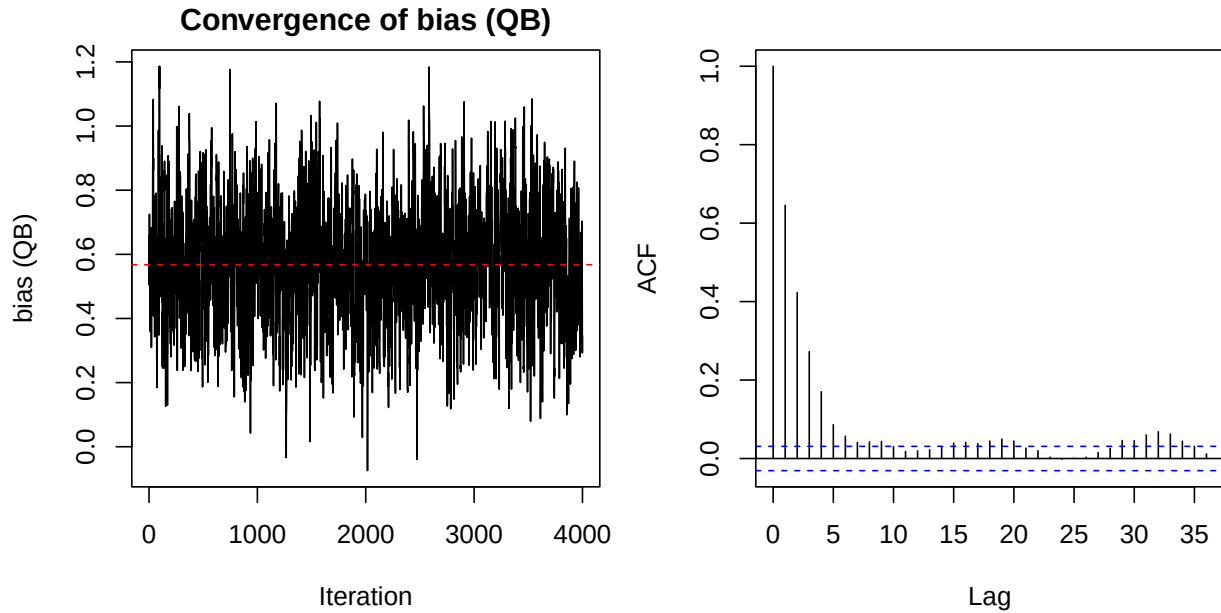
```
## WR test confusion (row = actual, col = predicted):
```

```
print(table(actual = pr$y_test, predicted = fits$WR$y_hat_test))
```

```
##      predicted
## actual 0  1
##      0  9 30
##      1  9 64
```

4. QB

```
S <- fits$QB$samples
pr <- fits$QB$prep
plot_chain_acf(S[, "bias"], parameter_name = "bias (QB)")
```



```
knitr::kable(posterior_quantile_table(S), digits = 3, caption = "QB: posterior summaries")
```

Table 4: QB: posterior summaries

	coefficient	posterior_mean	q1	q2	q3
bias	bias	0.568	0.443	0.570	0.695
Height	Height	0.491	0.141	0.475	0.842
Weight	Weight	0.699	0.452	0.681	0.942
40yd	40yd	-0.513	-0.667	-0.510	-0.358
Vertical	Vertical	0.329	0.180	0.328	0.477
passing_ATT	passing_ATT	-0.098	-0.386	-0.092	0.187
passing_TD	passing_TD	0.160	-0.088	0.155	0.402
passing_YDS	passing_YDS	0.981	0.635	0.963	1.326
passing_YPA	passing_YPA	0.185	-0.014	0.171	0.357

```
cat("QB test confusion:\n")
```

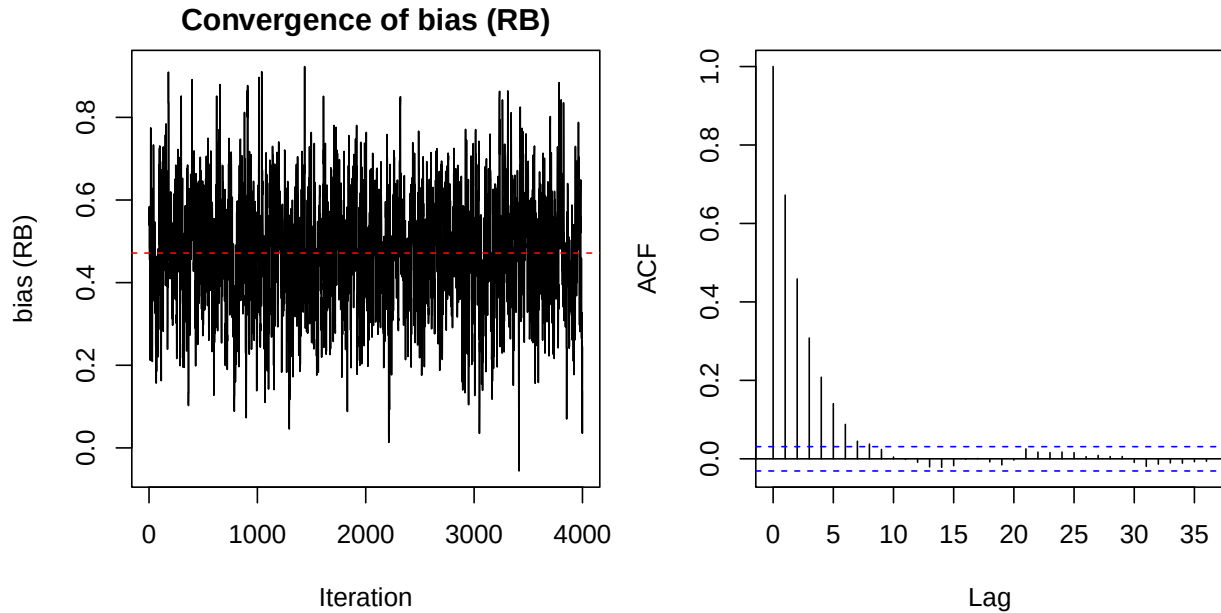
```
## QB test confusion:
```

```
print(table(actual = pr$y_test, predicted = fits$QB$y_hat_test))
```

```
##      predicted
## actual 0  1
##      0  4 13
##      1  6 20
```

5. RB

```
S <- fits$RB$samples
pr <- fits$RB$prep
plot_chain_acf(S[, "bias"], parameter_name = "bias (RB)")
```



```
knitr::kable(posterior_quantile_table(S), digits = 3, caption = "RB: posterior summaries")
```

Table 5: RB: posterior summaries

	coefficient	posterior_mean	q1	q2	q3
bias	bias	0.472	0.378	0.472	0.568
Height	Height	-0.207	-0.373	-0.201	-0.039
Weight	Weight	0.240	0.085	0.237	0.395
40yd	40yd	0.080	-0.024	0.082	0.190
Vertical	Vertical	0.228	0.120	0.229	0.338
rushing_CAR	rushing_CAR	0.040	-0.253	0.029	0.332
rushing_YDS	rushing_YDS	0.418	0.073	0.431	0.783
rushing_TD	rushing_TD	0.512	0.318	0.509	0.694
rushing_YPC	rushing_YPC	0.369	0.252	0.364	0.479

```
cat("RB test confusion:\n")
```

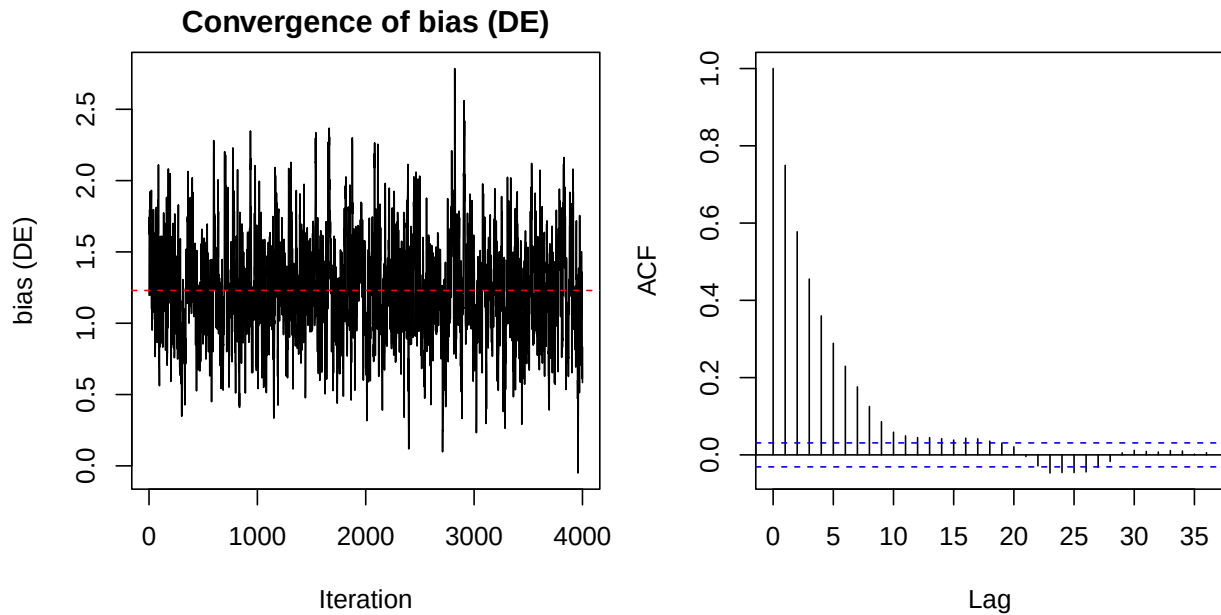
```
## RB test confusion:
```

```
print(table(actual = pr$y_test, predicted = fits$RB$y_hat_test))
```

```
##      predicted
## actual 0  1
##      0 12 17
##      1  8 31
```

6. DE

```
S <- fits$DE$samples
pr <- fits$DE$prep
plot_chain_acf(S[, "bias"], parameter_name = "bias (DE)")
```



```
knitr::kable(posterior_quantile_table(S), digits = 3, caption = "DE: posterior summaries")
```

Table 6: DE: posterior summaries

	coefficient	posterior_mean	q1	q2	q3
bias	bias	1.230	0.991	1.217	1.458
Height	Height	0.567	0.308	0.559	0.827
Weight	Weight	-0.231	-0.470	-0.232	0.016
40yd	40yd	0.281	0.037	0.283	0.524
Vertical	Vertical	-0.526	-0.777	-0.498	-0.244
Bench	Bench	0.543	0.298	0.540	0.774
defensive_SACKS	defensive_SACKS	0.868	0.419	0.847	1.309
defensive_TOT	defensive_TOT	-0.679	-1.129	-0.663	-0.225
defensive_TFL	defensive_TFL	0.573	0.071	0.565	1.085

```
cat("DE test confusion:\n")
```

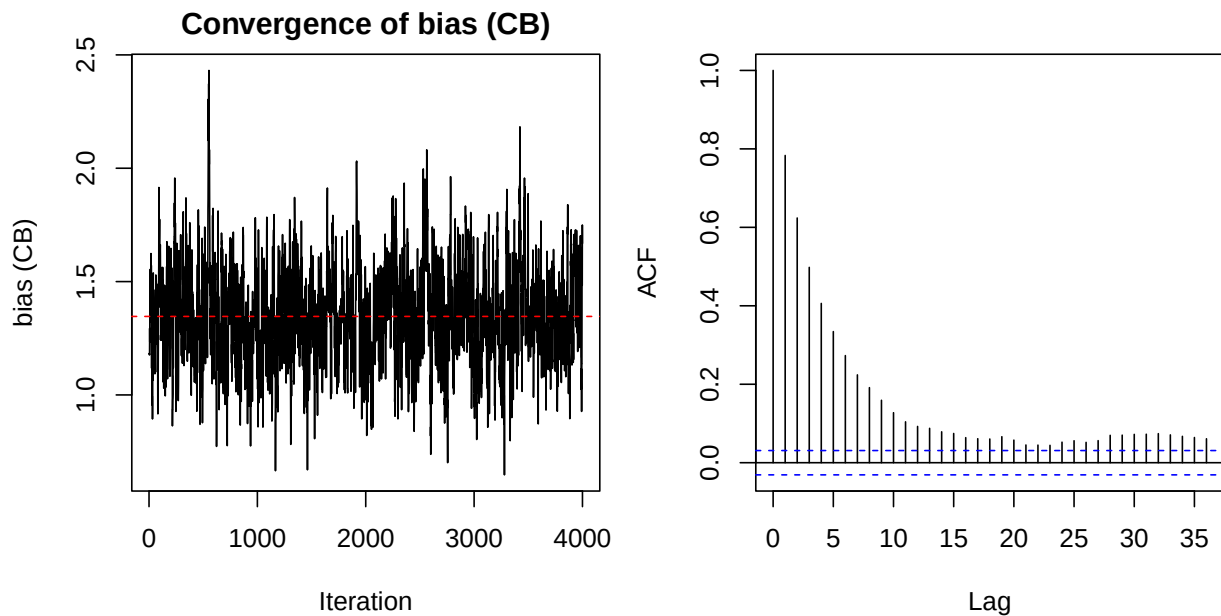
```
## DE test confusion:
```

```
print(table(actual = pr$y_test, predicted = fits$DE$y_hat_test))
```

```
##      predicted
## actual 0 1
##      0 0 4
##      1 2 9
```

7. CB

```
S <- fits$CB$samples
pr <- fits$CB$prep
plot_chain_acf(S[, "bias"], parameter_name = "bias (CB)")
```



```
knitr::kable(posterior_quantile_table(S), digits = 3, caption = "CB: posterior summaries")
```

Table 7: CB: posterior summaries

	coefficient	posterior_mean	q1	q2	q3
bias	bias	1.346	1.200	1.339	1.486
Height	Height	0.266	0.108	0.270	0.425
Weight	Weight	0.516	0.358	0.517	0.677
40yd	40yd	-0.241	-0.391	-0.224	-0.080
Vertical	Vertical	0.166	0.033	0.167	0.300
Bench	Bench	-0.024	-0.163	-0.023	0.121
defensive_PD	defensive_PD	0.271	0.053	0.267	0.491
defensive_TOT	defensive_TOT	-0.267	-0.498	-0.274	-0.040
interceptions_INT	interceptions_INT	0.620	0.476	0.611	0.760

```
cat("CB test confusion:\n")
```

```
## CB test confusion:
```

```
print(table(actual = pr$y_test, predicted = fits$CB$y_hat_test))
```

```
##      predicted
## actual 0  1
##      0  5 12
##      1  0 26
```